

```

1  /-
2  Copyright (c) 2026 Filippo A. E. Nuccio. All rights
   reserved.
3  Released under Apache 2.0 license as described in the file
   LICENSE.
4  Authors: Filippo A. E. Nuccio
5  -/
6
7  module
8
9  public import LFTCM2026.Preliminaries
10
11 /- # Structures and Classes:
12 Before embarking on today's class, let's see how things
   might *go wrong*, so that the design
13 choices will hopefully become clearer. -/
14
15 section Examples
16
17 -- ### Some crazy stuff
18
19
20 /- `Lean` is not so stupid after all, it understands that
   metric spaces have a topology...
21 but how is this possible? -/
22 example {X Y : Type*} [MetricSpace X] [MetricSpace Y] [Group
   Y] [IsTopologicalGroup Y]
23   (f g h : X → Y) : Continuous f → Continuous g →
   Continuous h → Continuous (f * g / h) := by
24   intro hf hg hh
25   apply Continuous.div'
26   · apply Continuous.mul
27     · exact hf
28     exact hg
29   · exact hh
30   -- fun_prop
31   done
32
33 /- Can you understand the error message (I'm not asking
   whether you understand *why* you get
34 the error: just what it says). -/

```

```

35 lemma quotComm_lemma {G : Type*} [Group G] (N : Subgroup G)
   : CommGroup (G / N) := by
36   sorry
37   done
38
39 -- This is false, but at least it compiles
40 def quotComm_def {G : Type*} [Group G] (N : Subgroup G) :
   Group (G / N) := by
41   sorry
42   done
43
44 -- And what goes on here?!?
45 #print continuous_fst
46 example (X Y : Type*) [TopologicalSpace X] [TopologicalSpace
   Y] [TopologicalSpace (X × Y)] :
47   Continuous (fun p : X × Y ↦ p.1) := by
48   exact continuous_fst
49   done
50
51
52 end Examples
53
54 section Structures
55
56 -- # A wrong way to define mathematical structures
57 /- The leitmotiv here is that in "informal" mathematics, the
   way we aggregate data and conditions
58 on these data is somewhat irrelevant. We can equally well
   say that
59 * A group is a set `G` with a preferred element `e` and a
   binary operation such that...
60 * A group law on a non-empty set `G` is the datum of a
   choice of some `e : G` and an operation ...
61 * In any of the above, we can require that `e * g = g`
   **and** `g * e = g` for every `g`; or just
62 one of the two, since one can prove that every right inverse
   is also a left inverse...
63 All these are considered *the same mathematical structure*:
   the choice is almost "typographical".
64
65 In a similar vein, we typically denote by `*` or `+` the
   operation, and in the first case we

```

```

66 sometimes write `e = 1`, in the second `e = 0`: but it is
    "clear" that the choice of the symbol
67 plays no role, and using `◆` and `e = §` would be good.
68
69 In Lean we need to do things differently.
70 -/
71
72 structure WrongGroup where
73   carrier : Type*
74   one : carrier
75   mul : carrier → carrier → carrier
76   inv : carrier → carrier
77   mul_one : ∀ (x : carrier), mul x one = x
78   one_mul : ∀ (x : carrier), mul one x = x
79   mul_assoc : ∀ (x y z : carrier), mul (mul x y) z = mul x
    (mul y z)
80   inv_mul_cancel : ∀ (x : carrier), mul (inv x) x = one
81
82 -- If this is how we play with products and inverses, we'd
    rather give up with `Lean` altogether...
83 lemma WrongGroup.inv_eq_of_mul {α : WrongGroup} (x y :
    α.carrier) :
84   α.mul x y = α.one → α.inv x = y := by
85     intro h
86     apply_fun (fun z ↦ α.mul (α.inv x) z) at h
87     -- use the `apply_fun` tactic to apply a function to
    both sides of a hypothesis
88     rw [α.mul_one, ← α.mul_assoc, α.inv_mul_cancel, α.one_mul]
    at h
89     exact h.symm
90     done
91
92 structure WrongSemigroup where
93   carrier : Type*
94   mul : carrier → carrier → carrier
95   mul_assoc : ∀ (x y z : carrier), mul (mul x y) z = mul x
    (mul y z)
96
97 -- A `lemma` saying that `a * ((b * c) * d) = (a * b) * (c *
    d)`
98 lemma assoc_mul (X : WrongSemigroup) (a b c d : X.carrier) :

```

```

99      X.mul a (X.mul (X.mul b c) d) = X.mul (X.mul a b) (X.mul
100      c d) := by
101      rw [X.mul_assoc]
102      done
103
104 /- If something is true in a Semigroup, it will stay so in a
105 group; so the above `lemma` should
106 still hold; yet...-/
107 lemma assoc_mul' (G : WrongGroup) (a b c d : G.carrier) :
108     G.mul a (G.mul (G.mul b c) d) = G.mul (G.mul a b) (G.mul
109     c d) := by
110     -- apply assoc_mul -- it does not work!
111     simp [G.mul_assoc]
112     done
113
114 /- Just to finish convincing ourselves that this `WrongXXX`
115 approach is *wrong*, let's try to
116 check that the usual addition makes `ℕ` into a
117 `(Wrong)Semigroup`. -/
118 def Nplus : WrongSemigroup where
119     carrier := ℕ
120     mul := (· + ·) -- or fun x y ↦ x + y
121     mul_assoc := add_assoc
122
123 example : Nplus.mul 1 1 = 2 := rfl
124 example : Nplus.mul (1 : Nplus) (1 : Nplus) = (2 : Nplus) :=
125     rfl
126 example : Nplus.mul (1 : ℕ) (1 : ℕ) = (2 : ℕ) := rfl
127
128 -- ## A right way to define mathematical structures
129
130 /- We'll focus on Groups in what follows, but just for the
131 sake of an example: their true theory
132 will be discussed in the lecture about Algebra. -/
133 #print Group
134 -- and right-clicking on it yields (the `_in_Cortona` avoids
135 Lean complaining this already exists)
136 structure Group_in_Cortona (G : Type*) extends DivInvMonoid
137     G where
138     protected inv_mul_cancel : ∀ a : G, a-1 * a = 1

```

```

132
133 #print DivInvMonoid
134
135
136 -- ## Our old friends  $\wedge$  and  $\leftrightarrow$  are also structures
137 #print And
138 #print Iff
139
140 example (P Q : Prop) : P  $\wedge$  Q  $\rightarrow$  ((P  $\rightarrow$  Q)  $\leftrightarrow$  (Q  $\rightarrow$  P)) := by
141   -- tauto
142   · rintro {hP, hQ}
143     constructor
144     · -- tauto
145       intro hPQ hQ'
146       exact hP
147     · exact fun _ _  $\mapsto$  hQ
148   done
149
150 /- But how to *remember* the names and list of all fields in
    a structure? Of course
151 one can have a look in the file where it is defined, but
    it's painful and might be very long...
152 -/
153 def DiscreteMetric (M : Type*) : MetricSpace M where
154   dist := sorry
155   dist_self := sorry
156   dist_comm := sorry
157   dist_triangle := sorry
158   edist_dist := sorry
159   uniformity_dist := sorry
160   cobounded_sets := sorry
161   eq_of_dist_eq_zero := sorry
162
163 /- Actually, try typing just
164
165 def DiscreteMetric (M : Type*) : MetricSpace M :=
166   _
167
168 and follow the bulb... 💡
169 -/
170

```

```

171 end Structures
172
173 noncomputable section Classes
174 open Classical
175
176 -- ### Some magic happens!
177
178 example (G : Type*) [Group G] (x y z : G) : x * y * z * 1 =
  x * (y * z) * 1 := by simp [mul_assoc]
179
180 -- actually `mul_assoc` does not only work for groups, yet
  `Lean` was happy using it! *Why?*
181 #check mul_assoc
182
183 -- Similarly...
184 lemma OneOne_Cortona {A : Type*} [Monoid A] (a : A) : a * 1
  * 1 = a := by simp
185
186 example {F : Type*} [NormedField F] (x : F) : x * 1 * 1 = x
  := by
187   exact OneOne_Cortona x
188   done
189
190 /- `Classes` are special `structures`, for which certain
  terms are stored in a database.
191
192 Each `class` type contains a *preferred* or a *canonical*
  term, declared using the keyword
193 `instance` rather than `def`; and this term gets registered
  in the database to be accessible
194 whenever needed. For example, we want that the terms of type
  `Field  $\mathbb{R}$ ` or `TopologicalSpace  $\mathbb{C}$ `,
195 representing respectively a field structure on  $\mathbb{R}$  and a
  topological structure on  $\mathbb{C}$ , are
196 found automatically, and are always what we expect them to
  be.
197
198 *Warning*: often, `Classes` have parameters, so if `G` and
  `H` are types, `Group G` and `Group H`
199 are different types! In particular, you don't expect the
  "type" `TopologicalSpace` to have an

```

```

200 instance (indeed, `TopologicalSpace` is not a type, it's a
    collection of types!): rather, you
201 expect the type `TopologicalSpace  $\mathbb{R}$ `, that contains all
    possible topologies on the set ` $\mathbb{R}$ ` to
202 have an instance, and likely you want it to be the Euclidean
    topology.
203
204 Classes also enable class type inference, constructing a
    term of a certain class given a term
205 of a "parent" one. -/
206
207
208 #synth Group  $\mathbb{Z}$ 
209 #synth AddGroup  $\mathbb{Z}$ 
210 #synth AddGroup ( $\mathbb{Z} \times \mathbb{Z}$ )
211 #synth TopologicalSpace  $\mathbb{C}$ 
212
213 def  $\mathbb{C}_{\text{Cortona}}$  :=  $\mathbb{C}$ 
214 #synth TopologicalSpace  $\mathbb{C}_{\text{Cortona}}$ 
215
216 #print One
217 #synth One  $\mathbb{Z}$ 
218
219 example : AddGroup ( $\mathbb{Z} \times \mathbb{Q}$ ) := inferInstance
220
221 -- Since finding instances is "automatic", the search can be
    used to create new ones:
222 variable (X Y : Type) [TopologicalSpace X] [TopologicalSpace
    Y] in
223 #synth TopologicalSpace ( $X \times Y$ )
224
225 variable (X Y : Type) [MetricSpace X] [MetricSpace Y] in
226 #synth TopologicalSpace ( $X \times Y$ ) -- Lean went from
    `MetricSpace` to `TopologicalSpace` automatically.
227
228 variable (X : Type) [MetricSpace X] in
229 #synth MetricSpace ( $X \times \mathbb{N}$ )
230 #synth MetricSpace  $\mathbb{N}$ 
231
232 /- ...but not all the time, especially if we don't have a
    canonical candidate: in the following,

```

```

233 you can think at  $((i : \iota) \rightarrow X\ i) = \prod (i : \iota), X\_i$  and  $(i$ 
     $: \iota) \rightarrow \text{MetricSpace } (X\ i)$  as a metric
234 structure on each of the  $X\_i$ 's.
235 -/
236 variable ( $\iota : \text{Type}^*$ ) ( $X : \iota \rightarrow \text{Type}^*$ ) [ $(i : \iota) \rightarrow \text{MetricSpace}$ 
    ( $X\ i$ )]
237 #synth MetricSpace  $((i : \iota) \rightarrow X\ i)$  -- indeed,  $L_p$  spaces
    exist for all  $p$ !
238
239 variable ( $\iota : \text{Type}^*$ ) ( $X : \iota \rightarrow \text{Type}^*$ ) [ $(i : \iota) \rightarrow \text{Group } (X\ i)$ ]
240 #synth Group  $((i : \iota) \rightarrow X\ i)$ 
241
242 instance CortonaMetric ( $\iota : \text{Type}^*$ ) ( $X : \iota \rightarrow \text{Type}^*$ ) [ $(i : \iota)$ 
     $\rightarrow \text{MetricSpace } (X\ i)$ ] :
243   (MetricSpace  $((i : \iota) \rightarrow X\ i)$ ) where
244     dist x y := if x = y then 0 else 1
245     dist_self x := by grind
246     dist_comm x y := by grind
247     dist_triangle x y z := by grind
248     edist_dist x y := by split_ifs <=> simp_all
249     eq_of_dist_eq_zero {x} {y} := by grind
250
251 variable ( $\iota : \text{Type}^*$ ) ( $X : \iota \rightarrow \text{Type}^*$ ) [ $(i : \iota) \rightarrow \text{MetricSpace}$ 
    ( $X\ i$ )]
252 #synth MetricSpace  $((i : \iota) \rightarrow X\ i)$ 
253
254
255 example {A :  $\text{Type}^*$ } [AddGroup A] (x y : A) :  $x + y + 0 = x +$ 
    y := by
256   simp only [add_zero] -- when does  $\text{add\_zero}$  hold?
257   done
258
259 #print HAdd
260 -- @[inherit_doc] infixl:65 " + " => HAdd.hAdd
261
262
263 -- What's going on here?
264 example (G :  $\text{Type}^*$ ) [Group G] [CommGroup G] (g : G) :  $1 * g$ 
    = g := by
265   rw [one_mul]
266   done
267

```



```

268 instance : Add Bool where
269   add b1 b2 := b1 && b2
270
271 example : true + false = false := by rfl
272
273 end Classes
274
275 -- # Exercises
276
277 section Exercises
278
279 -- **Exercise**
280 -- Do you understand why the first of the next two lines
281 -- compiles while the second
282 -- throws an error?
282 example (M : Type*) (α : Monoid M) : (1 : M) = (1 : M) :=
  rfl
283 example (α : Type*) (M : Monoid α) : (1 : M) = (1 : M) :=
  rfl
284 /- **Sol.:** The second equality does not make sense because
285 `M` is a structure, so a collection of
286 many fields, not a type that can/cannot contain a term like
287 `1`. -/
288
289 open Function
290
291 /- **¶ Exercise**
292 Re-experience the pain of playing with *wrongly-defined*
293 groups. -/
294 lemma WrongGroup.mul_inv_cancel {α : WrongGroup} (x :
  α.carrier) :
295   α.mul x (α.inv x) = α.one := by
296   rw [← α.inv_mul_cancel (α.inv x), α.inv_eq_of_mul _ _
  (α.inv_mul_cancel x)]
297   done
298
299 /- **¶ Exercise**
300 Prove that in every additive group, the intersection of two
301 normal subgroups is normal:
302 even if you find a one-line proof, try to produce the whole
303 term. For reasons to be explained later,
304 the intersection is written `⋂` and types as `⋂inf`.

```

```

300
301 Since we haven't discussed yet
302 what a subgroup is, let alone a normal one, it can be useful
303 to -/
304 #print Subgroup.Normal --it's a class, so in particular a
305 structure!
306
307 example (A : Type*) [AddGroup A] (H K : AddSubgroup A) :
308   H.Normal → K.Normal → (H ∩ K).Normal := by
309   -- apply AddSubgroup.normal_inf_normal
310   intro hH hK
311   constructor
312   rintro n ⟨hnH, hnK⟩ g
313   exact ⟨hH.conj_mem n hnH g, hK.conj_mem n hnK g⟩
314   done
315
316 /- **¶ Exercise**
317 You'll soon see how `Mathlib` defines topological spaces.
318 For this exercise, we're implementing
319 a toy model: a structure that I call *Cortological* spaces,
320 where we just ask for the datum of a
321 collection of "opens", satisfying no intersection/union
322 property but just obeying the requirement
323 that the empty set and the universal set belong to the
324 collection. -/
325
326 -- **1** Define the structure of a Cortological space on a
327 type `X`: should this be a class?
328 /- *Sol.* : Yes, it should be a class, and here it is. -/
329 class CortologicalSpace (X : Type) where
330   opens : Set (Set X)
331   univ : Set.univ ∈ opens
332   empty : ∅ ∈ opens
333
334 /- **2** Put a Cortological structure on `ℕ`, declaring that
335 a non-empty set is "open" if it
336 contains all elements larger than a given bound, and then
337 prove that the intersection of two opens
338 is open:-/
339 /- *Sol.* -/
340 def contains_arbitrarily_large (S : Set ℕ) : Prop := ∃ N, ∀
341   a, N ≤ a → a ∈ S

```

```

332
333 instance : CortologicalSpace ℕ where
334   opens := {∅} u Set.ofPred contains_arbitrarily_large
335   univ := by simp [contains_arbitrarily_large]
336   empty := by simp
337
338 open CortologicalSpace in
339 example (X Y : Set ℕ) : X ∈ opens → Y ∈ opens → X ∩ Y ∈
  opens := by
340   intro hX hY
341   -- rcases (Set.mem_union _ _ _).mpr hX with _ | ⟨N, hN⟩
342   -- · simp_all
343   -- rcases (Set.mem_union _ _ _).mpr hY with _ | ⟨M, hM⟩
344   -- · simp_all
345   -- right
346   -- use max M N
347   -- intro a ha
348   -- exact ⟨hN a (by grind), hM a (by grind)⟩
349   rcases (Set.mem_union _ _ _).mpr hX, (Set.mem_union _ _
  _).mpr hY with
350     ⟨_ | ⟨N, hN⟩, _ | ⟨M, hM⟩⟩ <;> try simp_all
351   right
352   use max M N
353   grind
354   done
355
356
357 open CortologicalSpace
358 /- **3** Make sure that if `X` and `Y` are both
  `CortologicalSpaces`, Lean automatically puts a
359 structure of `CortologicalSpace` on `X × Y`: I'm not asking
  any relation among this structure
360 and the starting structures whatsoever... (remember that
  this is a *toy model*!): -/
361
362 instance (X Y : Type) [CortologicalSpace X]
  [CortologicalSpace Y] : CortologicalSpace (X × Y) where
363   opens := {·.univ } u {∅}
364   univ := by simp
365   empty := by simp
366

```

```

367 /- **4** Define the structure of punctured cortological
spaces and discuss advantages and
368 disadvantages of making it a class (think at the product of
two punctured Cortological spaces). -/
369 /- *Sol.* -/
370 class PuncturedCortologicalSpace (X : Type) extends
CortologicalSpace X where
371   pt : X
372
373 /- Against the choice of making it a class, there is the
remark that many Cortological Spaces can be
374 "punctured" in several ways, and we don't want Lean to
automatically pick up a point; on the other
375 hand, if we make it a class, we can automatically deduce a
structure of
376   `PuncturedCortologicalSpace X × Y` by -/
377 open PuncturedCortologicalSpace in
378 instance (X Y : Type) [PuncturedCortologicalSpace X]
[PuncturedCortologicalSpace Y] :
379   PuncturedCortologicalSpace (X × Y) where
380     pt := (pt, pt)
381
382 /- **¶ Exercise**
383 Given a multiplicative group `G` and an additive group `A`,
what is the right way of putting a
384 multiplicative structure on `G × A` where `(g, a) * (h, b) =
(g * h, a + b)`?
385 **HINTS:** This might be hard: a suggestion is to do it in
steps, first defining the multiplication
386 just as a function, then definining a `Mul` instance, then
providing a trivial lemma saying
387 how this multiplication behaves, and then providing the
final instance (where you actually don't
388 need to provide *all* fields, as some can be deduced
automatically: try to comment them to see when
389 `Lean` complains).
390 -/
391
392 /- *Sol.* -/
393 def AddMul_mul {G A : Type *} [Group G] [AddGroup A] : (G ×
A) → (G × A) → (G × A) :=
394   fun ⟨g, a⟩ ⟨h, b⟩ ↦ ⟨g * h, a + b⟩

```

```

395
396 instance {G A : Type*} [Group G] [AddGroup A] : Mul (G × A)
    where
397     mul := AddMul_mul
398
399 lemma AddMul_mul_def {G A : Type*} [Group G] [AddGroup A] (g
    h : G) (a b : A) :
400     (⟨g, a⟩ : G × A) * ⟨h, b⟩ = ⟨g * h, a + b⟩ := rfl
401
402 instance {G A : Type*} [Group G] [AddGroup A] : Group (G ×
    A) where
403     mul := AddMul_mul
404     mul_assoc := by
405         rintro ⟨g, a⟩ ⟨h, b⟩ ⟨k, c⟩
406         grind [AddMul_mul_def]
407         done
408     one := ⟨1, 0⟩
409     one_mul := by
410         rintro ⟨g, a⟩
411         rw [AddMul_mul_def]
412         simp only [Prod.mk.injEq, mul_eq_right, add_eq_right]
413         constructor <=> rfl
414         done
415     mul_one a := by
416         rw [AddMul_mul_def]
417         ext <=> simp <=> rfl
418         done
419     inv := fun ⟨g, a⟩ ↦ ⟨g-1, -a⟩
420     inv_mul_cancel := by
421         rintro ⟨g, a⟩
422         rw [AddMul_mul_def]
423         simp only [inv_mul_cancel, neg_add_cancel]
424         rfl
425         done
426
427
428 end Exercises
429

```